UNIVERSIDAD DE EL SALVADOR
Facultad Multidisciplinaria De Occidente
Departamento de Ingeniería y Arquitectura
Ingeniería en Desarrollo de Software

Asignatura:

DESARROLLO Y REUTILIZACIÓN DE SOFTWARE

Ciclo X / Quinto Año

Tema:

Proyecto de Asignatura:
-Documento Tecnico-
Tema N°5 : API RESTful y Singleton

Coordinadora de Cátedra:
Ing. Dinora Virginia de Melara

Tutora GT02:
Ing. Dinora Virginia de Melara

Alumnos:
Grupo N°2

Josué Edenilson Martínez Flores -- MF22036
Jose Arnoldo Landaverde Gomez -- LG22018
Rodrigo Vladimir Cruz Cortez -- CC22070
Nelson Josué Morán Mata -- MM19037
Axel Rodrigo Juarez Quevedo -- JQ22003

Fecha:
25/10/2026

Ingeniería en DESARROLLO DE SOFTWARE

PROPÓSITO DEL PROYECTO

El Proyecto es una API REST desarrollada con ASP.NET Core Web API sobre .NET 10. Su objetivo es administrar la información de empleados pertenecientes a las áreas de Ventas, Producción, Transporte y Marketing.

La API permite crear, consultar, actualizar y eliminar empleados mediante peticiones HTTP. Cada empleado se representa en formato JSON y contiene los siguientes datos:

- Id: identificador numérico único.
- Nombre: nombre del empleado.
- Cargo: puesto desempeñado por el empleado.
- Área: área a la que pertenece. Las opciones válidas son Ventas, Producción, Transporte y Marketing.

El proyecto no utiliza una base de datos ni Entity Framework Core. En su lugar, conserva los datos en el archivo empleados.json ubicado en la raíz del proyecto. Esto permite mantener una solución sencilla y adecuada para el propósito académico, sin perder los datos al detener y volver a iniciar la aplicación.

ESTRUCTURA Y RESPONSABILIDADES DE LOS COMPONENTES

Models/Empleado.cs

Contiene la clase Empleado, que representa el modelo de datos principal de la API. Sus propiedades son Id, Nombre, Cargo y Area. Los campos Nombre, Cargo y Area tienen validación Required; gracias al atributo ApiController, si falta alguno de estos valores, ASP.NET Core responde automáticamente con un error 400 Bad Request.

Controllers / EmpleadosController.cs

Es el punto de entrada HTTP de la API. Define la ruta base api/empleados y recibe las solicitudes CRUD. El controller no administra directamente las colecciones ni el archivo JSON. En su lugar, recibe IEmpleadoService mediante inyección de dependencias y delega en ese servicio las operaciones de consulta, creación, modificación y eliminación.

El controller mantiene responsabilidades simples:

- Interpreta las peticiones HTTP y sus parámetros.
- Valida que el área indicada sea permitida.
- Devuelve códigos HTTP apropiados, por ejemplo 200, 201, 204, 400 y 404.
- Entrega la gestión de datos al servicio.

Services/IEmpleadoService.cs
Define el contrato que debe cumplir el servicio de empleados. Incluye los métodos para obtener empleados, buscar por Id, crear, actualizar, eliminar y validar áreas. El controller depende de esta interfaz y no de una clase concreta. Esta separación reduce el acoplamiento y facilita cambiar la implementación del servicio en el futuro sin modificar las rutas de la API.

Services/EmpleadoSingletonService.cs
Implementa IEmpleadoService y concentra la lógica de negocio y persistencia. Es la clase registrada como Singleton de la aplicación. Conserva los empleados en memoria mientras la API está activa y sincroniza los cambios con empleados.json.

Program.cs
Configura la aplicación. Registra los controllers, Swagger y el servicio de empleados. La instrucción AddSingleton<IEmpleadoService, EmpleadoSingletonService>() es la que indica al contenedor de dependencias que debe crear una sola instancia de EmpleadoSingletonService para toda la ejecución de la API.

empleados.json
Es el archivo de persistencia local. Al iniciar el servicio, se leen los empleados almacenados en este archivo. Después de crear, actualizar o eliminar un empleado, se vuelve a guardar su contenido. Si el archivo no existe, el servicio crea uno con una lista vacía.

PATRONES Y PRINCIPIOS UTILIZADOS

El patrón principal del proyecto es Singleton. No se crea manualmente una instancia estática en el código del controller. ASP.NET Core administra el ciclo de vida del servicio mediante el contenedor de inyección de dependencias:

AddSingleton<IEmpleadoService, EmpleadoSingletonService>();

Con este registro, todas las instancias de EmpleadosController utilizan el mismo EmpleadoSingletonService. Por lo tanto, existe una fuente centralizada de datos en memoria para toda la aplicación. Esta decisión evita que cada solicitud cree una lista independiente de empleados.

Inyección de dependencias:
EmpleadosController recibe IEmpleadoService en su constructor. El framework entrega automáticamente la instancia Singleton registrada en Program.cs. Esta técnica permite que el controller dependa de una abstracción y no de detalles internos de almacenamiento. También mejora la legibilidad y permite reemplazar el servicio por otra implementación si más adelante se utiliza una base de datos u otra fuente de datos.

Separación de responsabilidades:
El proyecto aplica una separación sencilla de responsabilidades:

- El modelo Empleado describe los datos y sus validaciones básicas.
- El controller se ocupa de HTTP, rutas y respuestas.
- El servicio contiene reglas de negocio, almacenamiento, lectura y escritura del archivo JSON.
- Program.cs configura las dependencias y el pipeline de ASP.NET Core.

Esta organización no agrega capas innecesarias, pero evita concentrar toda la lógica en el controller.

Manejo seguro de concurrencia:
El servicio utiliza ConcurrentDictionary<int, Empleado> para almacenar los empleados en memoria. También usa SemaphoreSlim para controlar el acceso concurrente a las operaciones de lectura, modificación y escritura del archivo. Esto disminuye el riesgo de que dos solicitudes simultáneas dejen datos inconsistentes.

Para generar los identificadores se utiliza Interlocked.Increment. De esta manera, la asignación del nuevo Id se realiza de forma atómica y se evita reutilizar un identificador cuando varias operaciones de creación coinciden en el tiempo.

Persistencia segura del archivo:
Al guardar información, el servicio primero escribe los datos en un archivo temporal con extensión .tmp y luego reemplaza empleados.json. Si ocurre un error durante una creación, actualización o eliminación, el servicio restaura el estado en memoria que existía antes de la operación y propaga el error. Este mecanismo reduce el riesgo de dejar un archivo principal incompleto.

Clonación de objetos:
El servicio devuelve copias de los objetos Empleado en vez de entregar directamente las instancias almacenadas. Así se evita que una modificación accidental sobre un objeto devuelto altere el estado interno del Singleton sin pasar por los métodos de actualización definidos.

OPERACIONES DISPONIBLES

GET /api/empleados
Obtiene todos los empleados registrados. Devuelve 200 OK y una colección JSON. La operación se realiza de forma asíncrona a través del servicio.

GET /api/empleados/{id}
Busca un empleado por su identificador. Devuelve 200 OK con el objeto encontrado. Si el Id no existe, devuelve 404 Not Found.

POST /api/empleados
Crea un nuevo empleado. Recibe un JSON como el siguiente:

```
{
  "nombre": "Ana López",
  "cargo": "Gerente de Ventas",
  "area": "Ventas"
}
```

El Id no debe ser proporcionado por el cliente; el servicio asigna el siguiente Id disponible. Si el área es válida, responde 201 Created, incluye el empleado creado y agrega el encabezado Location para consultarlo posteriormente. Si el área no corresponde a una de las cuatro opciones permitidas, responde 400 Bad Request.

PUT /api/empleados/{id}
Actualiza Nombre, Cargo y Area del empleado indicado. Recibe un JSON con los datos actualizados. Si la actualización se realiza correctamente, devuelve 204 No Content. Si el Id no existe, responde 404 Not Found. Si el área es inválida, responde 400 Bad Request.

DELETE /api/empleados/{id}
Elimina el empleado indicado. Si existe, devuelve 204 No Content y guarda el cambio en empleados.json. Si no existe, devuelve 404 Not Found.

RESPUESTAS HTTP PRINCIPALES

200 OK: consulta realizada correctamente.
201 Created: empleado creado correctamente.
204 No Content: actualización o eliminación realizada correctamente; no se devuelve cuerpo de respuesta.
400 Bad Request: datos obligatorios faltantes o área no permitida.
404 Not Found: no existe un empleado con el Id solicitado.

EJECUCIÓN DEL PROYECTO

Requisitos previos:
- Tener instalado el SDK de .NET 10.
- Tener acceso a una terminal, Visual Studio o Visual Studio Code.

Desde la carpeta que contiene el archivo, ejecutar:

dotnet restore
dotnet run --launch-profile http

El perfil HTTP configurado usa la dirección:

http://localhost:5035

También existe un perfil HTTPS, cuya dirección principal es:

https://localhost:7048

Al iniciar con el entorno Development, Swagger queda habilitado. Para abrir la documentación interactiva se debe navegar a:

http://localhost:5035/swagger

Desde Swagger se puede desplegar cada ruta, seleccionar Try it out, completar los valores necesarios y ejecutar la petición. La interfaz muestra la respuesta HTTP, los encabezados y el JSON devuelto por la API.

Para verificar que el proyecto compila antes de ejecutarlo, se puede usar:

dotnet build

FLUJO GENERAL DE UNA SOLICITUD

1. El cliente, Swagger o una herramienta como Postman envía una petición a /api/empleados.
2. ASP.NET Core identifica la acción correspondiente en EmpleadosController mediante los atributos HTTP.
3. El controller valida los datos básicos y delega la operación en IEmpleadoService.
4. EmpleadoSingletonService procesa la operación sobre su instancia única en memoria.
5. Si existe un cambio, el servicio actualiza empleados.json.
6. El controller devuelve al cliente el código HTTP y, cuando corresponde, el objeto JSON resultante.

CONSIDERACIONES FINALES

La solución está orientada a un ejercicio académico y demuestra el uso práctico de ASP.NET Core Web API, controllers, inyección de dependencias y el patrón Singleton administrado por el framework. La persistencia mediante JSON mantiene el proyecto ligero y permite demostrar que los datos continúan disponibles después de reiniciar la aplicación.

En un proyecto de producción podrían añadirse autenticación, una base de datos, pruebas automatizadas, control de versiones de los datos y un sistema de registros más amplio. Estas características no se incluyeron para conservar el alcance solicitado y mantener una implementación clara.

Diagrama de flujo de la API Empleados

